

Estrutura de Repetição



Estrutura de Repetição

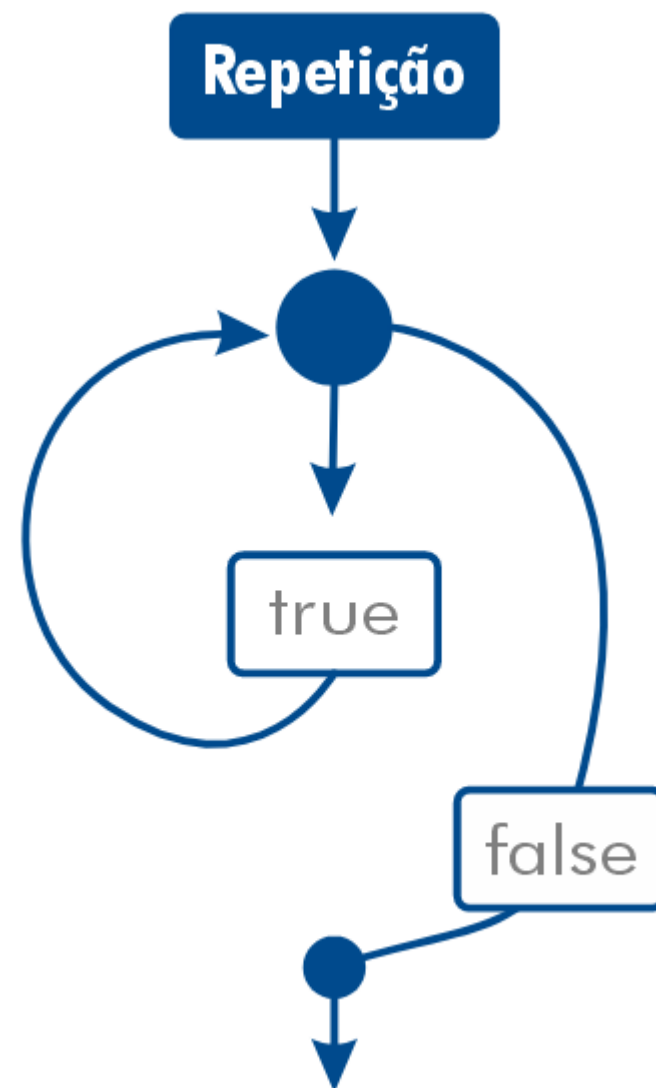
Uma estrutura de repetição em Python funciona da mesma maneira que uma estrutura em outras linguagens, como C e Java. A diferença básica é a sintaxe.

Agora que já falamos sobre as definições, vamos de fato entender como são a sintaxe e a forma dessas estruturas.

Para exemplo, vamos usar os loops **For** e **While**, que são os principais.

A diferença entre eles está na organização sintática e na maneira como se relacionam com a condição de término. É preciso saber exatamente como usar cada um e em quais cenários eles se aplicam.

Também falaremos sobre um problema comum quando se programa com loops, o loop infinito. Trataremos de estratégias para evitar essa situação e não atrasar o processo de desenvolvimento com erros fatais.



Estrutura de Repetição

Queremos que a frase abaixo apareça na tela 100 vezes.

```
print('Aula de repetição')  
print('Aula de repetição')  
print('Aula de repetição')  
  
# Isso continuaria até o 100...
```

Quantos prints seriam necessários?

Agora vamos analisar que é possível fazer isso apenas com 1 print.

Através de uma Estrutura de Repetição!

Tipos de Estrutura de Repetição

Assim como a declaração de variáveis ou os comandos de decisão, a sintaxe das estruturas de repetição depende da linguagem de programação usada.

- **For in**
 - **For in range**
- Na linguagem Python, ela é utilizada para percorrer elementos em sequência, como uma **string**, uma **lista**, uma **tupla** ou **objetos** iteráveis.
- **While**
 - **While else**
 - **While true**
- Estruturas de repetição permitem uma forma eficiente e organizada de lidar com problemas que envolvem repetição, tornando o código mais legível, fácil de manter e escalável.

- Anteriormente, aprendemos sobre desvios condicionais (`if`, `elif`, `else`), que servem para executar diferentes blocos de código com base em uma condição.
- Já as estruturas de repetição são blocos de código que permitem executar um conjunto de instruções várias vezes de forma **automática**, economizando **tempo** e **esforço**.
- Diferença-chave: Enquanto os desvios condicionais são executados **uma única vez** quando uma condição é verdadeira, as estruturas de repetição continuam executando o mesmo bloco de código **várias vezes** até que a condição de parada seja atingida.

Estrutura for



- **For Loop** é usado para iterar sobre uma sequência de elementos, como uma lista, tupla, dicionário ou até mesmo uma string. Ele percorre os itens um por um e executa um bloco de código para cada item.
- O **loop for** "itera" sobre cada elemento de uma coleção, como uma lista ou string. Ou seja, ele percorre cada item, executando um código para cada um deles.

Sintaxe:

```
for variavel in sequencia:  
    # Código a ser executado para cada item
```

- Aqui, variável assume, em cada iteração, o valor de um elemento da sequência. O bloco de código dentro do loop é executado uma vez para cada valor da sequência.

Estrutura de Repetição – for in

Sintaxe do for em Python

```
for item in conjunto_de_itens:  
    print(item)
```

item: corresponde a cada elemento presente na variável que permite a iteração;

conjunto_de_itens: pode ser uma lista, uma string, uma tupla, um dicionário ou um objeto que permita iterações.

In: String de associação, que verifica se algum item está/existe dentro de um conjunto de elementos.

Iteração sobre uma String

Neste caso, o loop `for` percorre cada letra da string e a imprime. Isso é útil quando queremos trabalhar com strings de forma detalhada.

```
palavra = 'Python'

for letra in palavra:
    print(letra)
```

Console

P
y
t
h
o
n

Função enumerate()

- A função `enumerate()` adiciona um contador a uma sequência iterável (como listas, tuplas, strings) e retorna um objeto enumerado, que pode ser usado diretamente em loops `for`.
- O `enumerate()` permite que você acesse tanto o índice quanto o valor dos itens da sequência ao iterar sobre ela.

```
enumerate(iteravel, start=0)
```

- `iterável`: A sequência sobre a qual você deseja iterar (lista, tupla, string, etc.).
- `start`: O valor inicial do contador (opcional, o padrão é 0).

Iteração sobre uma Lista usando `enumerate()`

Neste exemplo, a variável `fruta` assume o valor de cada item da lista `frutas` em cada iteração, e o `print()` exibe uma frase para cada fruta.

```
frutas = ['maçã', 'banana', 'laranja', 'uva']

for indice, fruta in enumerate(frutas, start=1):
    print(f"{indice}: {fruta}")
```

Console

```
1: maçã
2: banana
3: laranja
4: uva
```

Explicação

1. Lista de Frutas: A lista `frutas` contém quatro elementos.
2. `enumerate()` no Loop `for`: A função `enumerate()` retorna pares contendo o índice e o valor de cada elemento da lista.
3. Impressão de Índice e Valor: Para cada iteração, o índice da fruta e o nome da fruta são impressos.
4. `start=1`: começando a numeração com 1:

Exemplo de aplicação

Impressão do peso de 5 pessoas.

```
for pessoa in range(1, 6):  
    peso = float(input(f'Peso da {pessoa}ª pessoa: '))
```

Console

```
Peso da 1ª pessoa: 50  
Peso da 2ª pessoa: 62  
Peso da 3ª pessoa: 55  
Peso da 4ª pessoa: 85  
Peso da 5ª pessoa: 102
```

Vantagem e desafios

- O loop `for` é ótimo para quando já sabemos quantas vezes queremos repetir uma operação ou quando queremos percorrer uma coleção pré-definida (como uma lista ou string).
- O `for` é uma das estruturas de repetição mais poderosas e frequentemente usadas em Python.
- Ele simplifica a manipulação de coleções e sequências.
- Ao longo da aula, vamos explorar mais exemplos do `for`, como seu uso com a função `range()`, que abordaremos a seguir.

Exemplo:

```
palavra = input("Digite uma palavra: ")  
  
for letra in palavra:  
    print(letra)
```

Console

```
Digite uma palavra: Python  
P  
y  
t  
h  
o  
n
```

Estrutura for com range()



Estrutura for com range()

- O for loop em Python é também frequentemente usado para iterar sobre intervalos numéricos. Isso é feito usando a função `range()` para gerar uma sequência de números que o loop irá percorrer.
- A função `range()` é usada para gerar uma sequência de números, que pode ser iterada pelo loop `for`. Ela permite controlar o **início**, o **fim** e o **passo** de uma sequência de números.
- Usar `range()` é ideal quando precisamos repetir uma operação um número conhecido de vezes, ou quando queremos contar ou pular números em uma sequência.

```
for contador in range(1, 10, 2):  
    print(contador)
```

Sintaxe da função range()

- **Início (start):** (opcional) O número inicial da sequência. Se não for especificado, começa em 0.
- **Parada (stop):** (obrigatório) O número que determina o fim da sequência (não incluído na sequência).
- **Incremento (step):** (opcional) O valor de incremento. O padrão é 1.

start, stop, step

```
for i range(      ,      ,      ):
    print(i)
```

`for i range`(início, parada, incremento)
 `print(i)`

Estrutura for com range() - simples

Aqui, range(5) gera uma sequência de 0 a 4. O loop for percorre cada número dessa sequência e o imprime.

```
for i in range(5): # irá iterar de 0 até 4
    print(i)
```

0
1
2
3
4

Console

```
for cont in range(5):
    print(cont, end=' ')
```

Console

0 1 2 3 4

Usando range() com Parâmetros Múltiplos

- Especificando o valor inicial (**start**):

Se quisermos começar a contagem em um número diferente de 0, podemos especificar o valor de início no **range()**.

Aqui, o loop começa em 2 e vai até 5 (o valor 6 não é incluído, pois o **stop** não é inclusivo).

```
for contador in range(2, 6):  
    print(contador)
```

2
3
4
5

Console

```
for contador in range(1, 11):  
    print(contador, end=' ')
```

1 2 3 4 5 6 7 8 9 10

Console

Usando range() com Parâmetros Múltiplos

- Especificando o valor de incremento (**step**):

Podemos definir o tamanho do passo (incremento) da sequência com o parâmetro step.

O `range(1, 10, 2)` cria uma sequência começando em 1, indo até 9 (não inclui o 10), com um passo de 2 (ou seja, pula um número entre as iterações).

```
for i in range(1, 10, 2):  
    print(i)
```

1
3
5
7
9

Console

Usando range() com Parâmetros Múltiplos

- Usando valores negativos em (step):

O valor de incremento pode ser negativo, o que nos permite criar sequências decrescentes.

O `range(10, 0, -2)` gera uma sequência que começa em 10 e diminui em passos de 2 até chegar a 2 (não inclui o 0).

```
for i in range(10, 0, -2):  
    print(i)
```

10
8
6
4
2

Console

Range() na prática - Tabuada

Para cada linha que estiver dentro do intervalo faça:

```
for linha in range(1, 2):
    print('Tabuada:', linha)

    for coluna in range(10):
        print(f'{linha} + {coluna} = {linha + coluna}')
```

O intervalo do primeiro for range começará com 1, e a parada obrigatória irá encerrar em $(2 - 1) = 1$.

Ou seja, só terá a tabuada do número 1.

O intervalo do segundo for começará com 0, parada obrigatória em $(10 - 1) = 9$.

Console

Tabuada: 1

1 + 0 = 1

1 + 1 = 2

1 + 2 = 3

1 + 3 = 4

1 + 4 = 5

1 + 5 = 6

1 + 6 = 7

1 + 7 = 8

1 + 8 = 9

1 + 9 = 10

linha

coluna

Range() na prática - Tabuada

A linha começará no 2
Parada n-1 = 3
Incremento em 1 em 1

```
for linha in range(2, 4, 1):  
    print('Tabuada:', linha)  
  
    for coluna in range(1, 10, 1):  
        print(f'{linha} x {coluna} = {linha * coluna}')
```

A coluna começará no 1
Parada n-1 = 9
Incremento de 1 em 1

Console

Tabuada: 2

2 + 1 = 3
2 + 2 = 4
2 + 3 = 5
2 + 4 = 6
2 + 5 = 7
2 + 6 = 8
2 + 7 = 9
2 + 8 = 10
2 + 9 = 11

Tabuada: 3

3 + 1 = 4
3 + 2 = 5
3 + 3 = 6
3 + 4 = 7
3 + 5 = 8
3 + 6 = 9
3 + 7 = 10
3 + 8 = 11
3 + 9 = 12

Exemplos

```
for num in range(1, 7):  
    valor = int(input('Digite o {} valor: '.format(num)))
```

Console

```
Digite o 1 valor: 2  
Digite o 2 valor: 52  
Digite o 3 valor: 35  
Digite o 4 valor: 45  
Digite o 5 valor: 64  
Digite o 6 valor: 54
```

```
soma = 0  
cont = 0  
  
for num in range(1, 7):  
    valor = int(input('Digite o {} valor: '.format(num)))  
  
    if valor % 2 == 0:  
        soma += valor # ou soma = soma + valor  
        cont += 1     # ou cont = cont + 1  
  
print('Você informou {} números e a soma é {}'.format(cont, soma))
```

Console

```
Digite o 1 valor: 1  
Digite o 2 valor: 2  
Digite o 3 valor: 3  
Digite o 4 valor: 4  
Digite o 5 valor: 5  
Digite o 6 valor: 6  
Você informou 3 números e a soma é 12.
```

```
print('Você informou {} números e a soma é {}'.format(cont, soma))
```

Exemplos: string + for range

```
# ↓ (strip) eliminar os espaços antes e depois
# ↓ (upper) para converter em maiúsculo

frase = str(input('Digite um texto: ')).strip().upper()

palavras = frase.split() # para dividir as palavras e criar uma lista
junto = ''.join(palavras) # para eliminar os espaços e juntar a lista

inverso = '' # ↓ última letra, ↓ primeira, ↓ voltando uma letra

for letra in range(len(junto) - 1, -1, -1): # para criar o inverso do texto
    inverso += junto[letra]

print('O texto fica: {}'.format(inverso))
```

```
print('O texto fica: {}'.format(inverso))
```

Console

```
inverso += junto[letra]
```

```
Digite um texto: renata
O texto fica: ATANER
```


Exemplos: string + for range

```
frase = str(input('Digite um texto: ')).strip().upper()
palavras = frase.split()
junto = ''.join(palavras)

# ↓ o início, ↓ sem pegar nada, ↓ começar de trás pra frente
inverso = junto[::-1]

print('O texto fica: {}'.format(inverso))
```

```
print('O texto fica: {}'.format(inverso))
```

Console

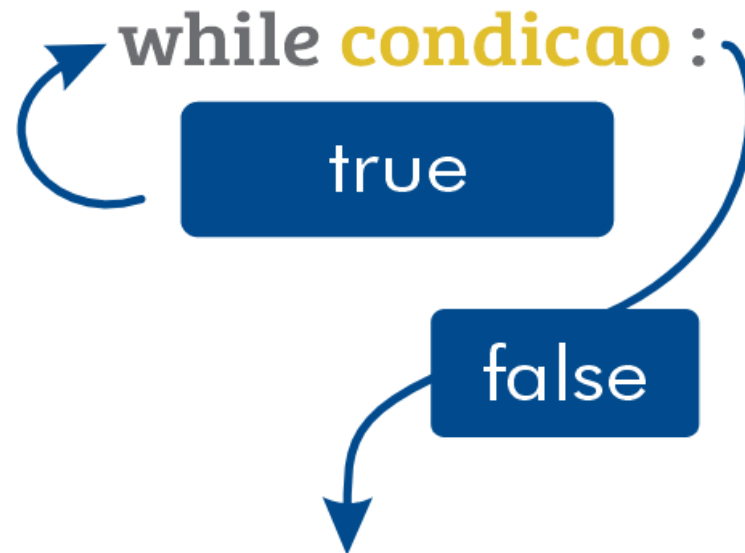
```
Digite um texto: renata
O texto fica: ATANER
```

Estrutura while



Loop While

- O loop while é uma estrutura de repetição que continua executando um bloco de código enquanto uma condição for verdadeira.
- Diferente do for, que geralmente é usado quando sabemos o número de repetições antecipadamente, o while é ideal para situações em que não sabemos quantas vezes o loop será executado. O loop pode continuar indefinidamente até que a condição seja satisfeita (ou seja, até que a condição se torne falsa).



Sintaxe do while

- Condição: É uma expressão booleana (que resulta em **True** ou **False**). Enquanto for **True**, o código dentro do loop será repetido.
- Bloco de código: O bloco de código será executado repetidamente até que a condição se torne **False**.

```
while condição:  
    # código a ser executado  
    # enquanto a condição for verdadeira
```

- Como funciona?

1. A condição é avaliada.

1. Se a condição for **True**, o bloco de código é executado.

2. Após o bloco ser executado, a condição é verificada novamente.

3. O ciclo continua até que a condição se torne **False**.

- O laço **while** executa um ou mais comandos enquanto a condição seja verdadeira.
- Precisamos de uma **variável de apoio(contador)** para indicar o ponto de parada da estrutura de repetição while.
Caso o programa não tenha incremento ficará em “**loop infinito**”.
- É necessário incrementar a contadora para que o loop encontre a parada.

Terminará no número 5

```
contadora = 1
while contadora <= 5:
    print(contadora)
    contadora = contadora + 1

Print('Saiu do laço!')
```

Console

```
1
2
3
4
5
```

Exemplo Simples de um Loop while

```
contador = 0

while contador < 5:
    print(contador)
    contador += 1
```

0
1
2
3
4

Console

Explicação:

- O loop começa com `contador = 0`.
- A condição é `contador < 5`, ou seja, o loop continuará enquanto o valor de `i` for menor que 5.
- A cada iteração, `contador` é incrementado em 1 (`contador += 1` ou `contador = contador + 1`).
- Quando `contador` atinge 5, a condição se torna `False` e o loop termina.

Diferença entre `for` e `while`

- O `for` é usado quando sabemos de antemão quantas vezes queremos que o loop se repita. Ele é ótimo para iterar sobre sequências ou usar contadores com `range()`.
- O `while` é utilizado quando não sabemos o número exato de iterações. Ele depende de uma condição que pode mudar dinamicamente, e o loop continuará até que essa condição se torne falsa.

```
# Usando for
for contador in range(5):
    print(contador)
```

```
# Usando while
contador = 0

while contador < 5:
    print(contador)
    contador += 1
```

0
1
2
3
4

Console

- Ambos os exemplos acima imprimem os números de 0 a 4, mas no `while`, o controle sobre a variável `contador` e a condição de parada é manual.

Exemplo prático com while

Somar Números até o Usuário Digitar Zero

- Vamos usar o **while** para somar números fornecidos pelo usuário até que ele digite zero.

Console

```
soma = 0

num = int(input("Digite um número (0 para sair): "))

while num != 0:
    soma += num
    num = int(input("Digite outro número (0 para sair): "))

print(f"Soma total: {soma}")
```

```
Digite um número (0 para sair): 0
Soma total: 0
```

```
Digite um número (0 para sair): 2
Digite outro número (0 para sair): 3
Digite outro número (0 para sair): 0
Soma total: 5
```

- Explicação: O loop **while** continua solicitando números até que o usuário insira o número 0. O valor digitado é somado à variável **soma** a cada iteração.

Exemplo simples com while

Somar Números até o Usuário Digitar Zero

- Vamos usar o `while` que o usuário digite um valor até que digite zero.

```
n = 1

while n != 0:
    n = int(input('Digite um valor: '))

print('FIM')
```

```
input(,FIW.)
```

Console

```
Digite um valor: 2
Digite um valor: 3
Digite um valor: 5
Digite um valor: 4
Digite um valor: 0
FIM
```

```
Digite um valor: 1
Digite um valor: 0
FIM
```

- Explicação: O loop `while` continua solicitando números até que o usuário insira o número 0, onde teremos um ponto de parada / condição de parada.

Exemplo simples com while

Perguntando se o usuário quer ou não continuar

- Vamos usar o **while** para perguntar se o usuário deseja sair ou continuar.

```
res = 'S'

while res == 'S':
    num = int(input('Digite um número: '))
    res = str(input('Deseja continuar? [S/N] ')).upper()

print('FIM')
```

```
butu(.fIW.)
```

Console

```
Digite um número: 2
Deseja continuar? [S/N] s
Digite um número: 5
Deseja continuar? [S/N] s
Digite um número: 6
Deseja continuar? [S/N] n
FIM
```

- Explicação: O loop **while** continua perguntando se o usuário deseja continuar ou não, enquanto é digitado 'S/s' continua, ao digitar 'N/n', chega no fim.

Exemplo práticos com while

Adivinhe o Número

- Um programa que usa `while` para permitir que o usuário tente adivinhar um número secreto.

Console

```
numero_secreto = 7

tentativa = int(input("Adivinhe o número secreto (1 a 10): "))

while tentativa != numero_secreto:
    print("Tente novamente!")
    tentativa = int(input("Adivinhe o número secreto (1 a 10): "))

print("Parabéns, você acertou!")
```

```
print("Parabéns, você acertou!")
```

```
Adivinhe o número secreto (1 a 10): 3
Tente novamente!
Adivinhe o número secreto (1 a 10): 5
Tente novamente!
Adivinhe o número secreto (1 a 10): 2
Tente novamente!
Adivinhe o número secreto (1 a 10): 7
Parabéns, você acertou!
```

- Explicação: O loop continua até que o usuário adivinhe corretamente o número secreto. O loop é interrompido quando a condição (`tentativa != numero_secreto`) se torna falsa.

Comandos auxiliares para estrutura de repetição



Break

- **Break:** É um comando de interrupção de um loop. Não importa o estado que se encontra a execução, ele parará o loop imediatamente.

```
while condicao:
    if alguma_condicao:
        break
    # código do loop
```

Exemplo:

```
contador = 0
while contador < 10:
    if contador == 5:
        break
    print(contador)
    contador += 1
```

- O loop continua enquanto **contador** for menor que 10, mas quando **contador** atinge 5, o comando **break** interrompe o loop.

Console

```
0
1
2
3
4
```

- **Continue**: O continue é semelhante ao break, no entanto, no lugar dele parar o loop por completo, ele apenas encerra os comandos que esteja abaixo dele e retorna para a próxima iteração.

```
while condicao:

    # Código antes do continue

    if condicao_para_pular:
        continue # Pula para a próxima iteração do loop

    # Código após o continue
```

```
# código após o continue
```

```
contador = 0
while contador <= 9:
    contador += 1
    if contador == 6:
        print('Número 6 não foi impresso')
        continue
    print(contador)
```

Irá pular o comando abaixo e
retornar para o laço novamente

Console

```
1
2
3
4
5
'Número 6 não foi
impresso'
7
8
9
10
```

OBS: Número 6 não foi impresso porque o continue voltou para uma nova iteração e descartou o print abaixo dele.

Continue

Exemplo:

- Vamos criar um programa que imprime números de 1 a 10, mas pula os números ímpares

```
contador = 1

while contador <= 10:

    if contador % 2 != 0: # Verifica se o número é ímpar
        contador += 1    # Incrementa contador para evitar loop infinito
        continue        # Pula para a próxima iteração se for ímpar

    print(contador)      # Imprime o número se for par
    contador += 1        # Incrementa contador
```

Console

2
4
6
8
10

- Explicação:**
 - Inicialização: A variável **i** é inicializada com 1.
 - Condição do Loop: O loop continua enquanto **i** for menor ou igual a 10.
 - Verificação de Ímpares: Se **i** for ímpar (**i % 2 != 0**), o código incrementa **i** e usa **continue** para pular a impressão e ir para a próxima iteração do loop.
 - Impressão de Números Pares: Se **i** for par, ele é impresso, e **i** é incrementado.

While Loop Infinito

Existe outro modo de while que executa um loop sem a necessidade de uma contadora. Neste caso, o while irá ser um loop infinito proposital.

```
while True:
    opcao = int(input('1 - Somar ou 2 - Sair: '))

    if opcao == 1:

        n1 = int(input('Informe o número: '))
        n2 = int(input('Informe outro número: '))

        soma = n1 + n2

        print(soma)
    else:
        print('Saiu do Laço!')
        break
```

Break para o loop proposital imediatamente

Cuidados com o Loop Infinito

- Um erro comum ao usar `while` é criar acidentalmente um loop infinito, que acontece quando a condição nunca se torna `False`. Isso faz com que o programa continue rodando indefinidamente.

```
contador = 0

while contador < 5:
    print(contador)
```

Aqui, o ' `contador` ' nunca é incrementado, então a condição '`contador < 5`' nunca será '`False`'.

- Como evitar: Lembre-se sempre de alterar a variável que controla a condição dentro do loop, para garantir que ele eventualmente pare.

Estrutura while else



Estrutura while/else

- A estrutura **while/else** combina um loop **while** com um bloco **else**, que é executado quando a condição do loop se torna falsa.
- Isso pode ser útil para realizar uma ação específica após a conclusão do loop, sem a necessidade de usar um **break**.

```
while condição:
    pass
    # bloco de código

else:
    # bloco de código a ser executado quando a condição se torna falsa
```

- **Como Funciona?**
 - 1.O loop **while** executa seu bloco de código enquanto a condição for verdadeira.
 - 2.Quando a condição se torna falsa, o bloco **else** é executado.
 - 3.Se o loop for interrompido com um **break**, o bloco **else** não será executado.

While else

else é um comando opcional no loop **while**

```
contadora = 1
while contadora <= 5:
    print(contadora)
    contadora += 1
else:
    print("Variável =", contadora)
    print("Final do loop while!")
```

→ Mesmo coisa que: `contadora = contadora + 1`

Só é executado quando a condição testada no loop **não** for verdadeira.

Console

```
1
2
3
4
5
Variável = 6
Final do loop while!
```

Exemplo Simples de while/else

Vamos criar um exemplo onde solicitamos ao usuário que adivinhe um número secreto. O loop continua enquanto o usuário não adivinhar corretamente e, ao final, uma mensagem será exibida usando o else.

```
numero_secreto = 7
tentativa = None

while tentativa != numero_secreto:
    tentativa = int(input("Adivinhe o número secreto (1 a 10): "))
else:
    print("Parabéns, você adivinhou o número!")
```

```
print("Parabéns, você adivinhou o número!")
```

Console

```
Adivinhe o número secreto (1 a 10): 3
Adivinhe o número secreto (1 a 10): 2
Adivinhe o número secreto (1 a 10): 7
Parabéns, você adivinhou o número!
```

Explicação:

1. **Inicialização:** O número secreto é definido como 7 e a variável `tentativa` é inicializada como `None`.
2. **Loop de Adivinhação:** O loop `while` continua pedindo ao usuário para adivinhar o número até que a `tentativa` seja igual ao `numero_secreto`.
3. **Bloco else:** Quando a condição do loop se torna falsa (ou seja, quando o usuário adivinha o número corretamente), a mensagem de parabéns é impressa.

Exemplo com break e else

Se o loop for interrompido com break, o bloco else não será executado. Veja um exemplo:

```
numero_secreto = 7
tentativa = 0
tentativas = 0
tentativas_maximas = 3

while tentativa != numero_secreto:
    tentativa = int(input("Adivinhe o número secreto (1 a 10): "))
    tentativas += 1

    if tentativas >= tentativas_maximas and tentativa != numero_secreto:
        print("Você excedeu o número máximo de tentativas.")
        break
else:
    print("Parabéns, você adivinhou o número!")
```

Console

```
Adivinhe o número secreto (1 a 10): 2
Adivinhe o número secreto (1 a 10): 6
Adivinhe o número secreto (1 a 10): 4
Você excedeu o número máximo de tentativas.
```

Explicação:

1. Tentativas Máximas: O usuário tem um número máximo de tentativas definido por tentativas_maximas.
2. Bloco break: Se o número máximo de tentativas for atingido, o loop é interrompido com break, e o bloco else não é executado.
3. Mensagens Diferentes: Se o usuário adivinhar corretamente, receberá uma mensagem de parabéns; caso contrário, uma mensagem informando que o número máximo de tentativas foi excedido.

Estrutura while true



- A estrutura `while True` é um loop que executa seu bloco de código indefinidamente porque a condição `True` nunca se torna falsa.
- Esse tipo de loop é frequentemente utilizado em situações onde você deseja que um bloco de código continue a ser executado até que uma condição interna (geralmente um `break`) seja satisfeita.

```
while True:  
    # bloco de código  
  
    if condição_para_parar:  
        break
```

Como Funciona?

- 1.O loop começa e a condição `True` é avaliada, sempre resultando em verdadeiro.
- 2.O bloco de código dentro do loop é executado repetidamente.
- 3.Quando uma condição específica é satisfeita (normalmente uma verificação de entrada do usuário ou outra lógica), o comando `break` é acionado, interrompendo o loop.

Exemplo While True

```
while True:
    calcular = input('1 - Para Somar ou 2 - Para Subtrair: ')
    if calcular == '1':
        print('Bem vindo a soma!')
        break

    elif calcular == '2':
        print('Bem vindo a subtração!')
        break

    else:
        print('Digitou número inválido! Tente novamente!')

        # Transformando em minúsculo e verificando a primeira letra
        sair = input('Sair do programa? [s]im ou n[ao]: ').lower().startswith('s')

        # O "is" é um comando lógico
        if sair is True:
            break

        else:
            continue
```

continue

Exemplo Simples

Vamos criar um programa que continua pedindo ao usuário para inserir um número até que ele insira 0, que servirá como condição de saída.

```
while True:
    num = int(input("Digite um número (0 para sair): "))
    if num == 0:
        print("Programa encerrado.")
        break # Interrompe o loop se o número for 0

    print(f"Você digitou: {num}")
```

Console

```
Digite um número (0 para sair): 2
Você digitou: 2
Digite um número (0 para sair): 0
Programa encerrado.
```

Explicação:

1. Loop Infinito: O loop while True inicia, e o código dentro dele será executado continuamente.
2. Entrada do Usuário: O programa pede ao usuário para digitar um número.
3. Condição de Saída: Se o usuário digitar 0, o comando break é acionado, o que interrompe o loop e exibe a mensagem de encerramento.
4. Impressão do Número: Se o número não for 0, ele é impresso.

Exemplo Avançado – Menu interativo

```
while True:
```

```
    print("\nMenu:")
    print("1. Adicionar")
    print("2. Remover")
    print("3. Sair")
```

```
    opcao = input("Escolha uma opção: ")
```

```
    if opcao == '1':
        print("Você escolheu Adicionar.")
```

```
        # Lógica para adicionar
```

```
    elif opcao == '2':
        print("Você escolheu Remover.")
```

```
        # Lógica para remover
```

```
    elif opcao == '3':
        print("Saindo do programa...")
```

```
        break # Interrompe o loop
```

```
    else:
        print("Opção inválida. Tente novamente.")
```

No console

Menu:

1. Adicionar
2. Remover
3. Sair

Escolha uma opção: 1
Você escolheu Adicionar.

Menu:

1. Adicionar
2. Remover
3. Sair

Escolha uma opção: 2
Você escolheu Remover.

Menu:

1. Adicionar
2. Remover
3. Sair

Escolha uma opção: 3
Saindo do programa...

Explicação:

1. Menu Repetido: O menu é exibido a cada iteração do loop.
2. Seleção de Opção: O usuário pode escolher uma opção. Se a opção for 3, o loop é interrompido com **break**.
3. Tratamento de Opção Inválida: Se o usuário digitar uma opção inválida, uma mensagem é exibida, e o menu é reapresentado.

Exercícios



Exercício 1

Crie um programa para que o usuário informe seu sexo [M/F].

```
# [0] pega apenas a primeira letra digitada
sexo = str(input('Informe seu sexo: [M/F] ')).strip().upper()[0]

while sexo not in 'MF':
    sexo = str(input('Dados inválidos. Informe seu sexo: [M/F] ')).strip().upper()[0]

print(f'Sexo {sexo} registrado com sucesso!')
```

Console

```
Informe seu sexo: [M/F] d
Dados inválidos. Informe o seu sexo: [M/F] k
Dados inválidos. Informe o seu sexo: [M/F] feminino
Sexo F registrado com sucesso!
```

Exercício 2

Crie um programa para que o usuário informe a quantidade de números que ele quiser e use um comando para finalizar.

```
num = 0
soma = 0

while True:
    num = int(input('Digite um número ou [00] pra sair: '))

    if num == 0:
        break

    soma += num

print(f'A soma dos números é {soma}.')
```

Console

```
Digite um número ou [00] pra sair: 10
Digite um número ou [00] pra sair: 5
Digite um número ou [00] pra sair: 5
Digite um número ou [00] pra sair: 00
A soma dos números é 20.
```

Exercício3

Crie um programa para criar qualquer tabuada e que finalize com números negativos.

```
while True:

    num = int(input('Qual a tabuada? '))

    if num < 0:
        break

    for contador in range(1, 11):

        print(f'{num} x {contador} = {num * contador}')

print('** PROGRAMA FINALIZADO **')
```

Console

```
Qual a tabuada? 3
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
```

Console

```
Qual a tabuada? 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
Qual a tabuada? -1
** PROGRAMA FINALIZADO **
```


Exercício 4

Usando a função range() e while, informe os números de 1 a 50.

range()

```
for contador in range(1, 51):  
    print(contador, end=' ')
```

while

```
contador = 1  
  
while contador <= 50:  
    print(contador, end=' ')  
  
    # Incrementa o contador em +1  
    contador += 1  
  
    # Também poderia ser:  
    # contador = contador + 1
```

Console

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50  
Process finished with exit code 0
```

Exercício 5

Faça um programa que peça uma nota, entre zero e dez. Mostre uma mensagem caso o valor seja diferente do intervalo pedido. E continue pedindo até que o usuário informe um valor válido. Use a estrutura de repetição **while**.

```
nota = float(input("Insira uma nota de 0 até 10: "))

while nota < 0 or nota > 10:
    nota = float(input("Nota inválida. Tente novamente: "))

print("Nota válida")
```

Console

```
Digite uma nota de 0 até 10: 2
Nota válida
```

```
Digite uma nota de 0 até 10: 11
Nota Inválida. Tente novamente: 56
Nota Inválida. Tente novamente: 12
Nota Inválida. Tente novamente: 10
Nota válida
```

Exercício 6

Faça um programa que peça uma nota, entre zero e dez. Mostre uma mensagem caso o valor seja diferente do intervalo pedido. E continue pedindo até que o usuário informe um valor válido.

Use a estrutura de repetição **while True**, or, as e **ValueError**.

E agora com uma mensagem caso o usuário digite um texto.

```
while True:
    try:
        # Solicita uma nota entre 0 e 10
        nota = float(input('Digite uma nota de 0 até 10: '))

        # Verifica se a nota está fora do intervalo
        while nota < 0 or nota > 10:
            nota = float(input('Nota inválida. Tente novamente: '))

        print('Nota válida')

        # Encerra o loop quando a nota for válida
        break

    # Trata erro caso o usuário digite texto
    except ValueError as erro:
        print('Apenas números e não letras')
```

Console

```
Digite uma nota de 0 até 10: 45
Nota Inválida. Tente novamente: 65
Nota Inválida. Tente novamente: a
Apenas números e não letra
Digite uma nota de 0 até 10: 5a
Apenas números e não letra
Digite uma nota de 0 até 10: 2
Nota válida
```

Exercício 7

Faça um programa que leia um nome de usuário e a sua senha. Não aceite a senha igual ao nome do usuário, caso isso ocorra, deve mostrar uma mensagem de erro e voltar a pedir as informações.

```
login = input("Login: ")
senha = input("Senha: ")

while login == senha:
    print("Sua senha deve ser diferente do login.")
    senha = input("Senha: ")

print("Cadastro aprovado")
```

Console

```
Digite o login: renata123
Digite a senha: renata123
Senha deve ser diferente do login
Digite a senha: 123renata
Senha aprovada
```

Exercício 8

Faça um programa que receba as seguintes informações do usuário: **nome**, **idade** e **salário**. Valide as seguintes informações:

- O **nome** deve ser maior que 3 caracteres, ou seja, o nome Ana, por exemplo, não passaria no laço de repetição. Dica: use o método **len()** para verificar o tamanho do nome.
- **Idade** deve estar entre 0 e 150.
- O **salário** não pode ser negativo.
- O programa só retornará os dados finais se todos os critérios forem atendidos, caso algum critério não passar, deve continuar o **loop** até o usuário acertar.
- Dica: Use o laço de repetição **while**, três vezes, um para cada validação.

Exercício 8

Console

```
Qual seu nome [minimo 4 caracteres]: Renata
Sua idade: 38
Salário: 1000
-----
Informações do usuário:
Nome: Renata, Idade: 38 e Salário: R$ 1000.0
-----
```

Console

```
Qual seu nome [minimo 4 caracteres]: 5
Seu nome deve ter mais que 3 caracteres: Renata
Sua idade: 500
Voce deve ter entre 0 e 150 anos: 38
Salário: -1
Informe um salário válido: 1000
-----
Informações do usuário:
Nome: Renata, Idade: 38 e Salário: R$ 1000.0
-----
```

Exercício 8

```
# Validação do nome
nome = input('Digite seu nome: ')
while len(nome) <= 3:
    nome = input('Nome inválido! Digite um nome com mais de 3 caracteres: ')

# Validação da idade
idade = int(input('Digite sua idade: '))
while idade < 0 or idade > 150:
    idade = int(input('Idade inválida! Digite uma idade entre 0 e 150 anos: '))

# Validação do salário
salario = float(input('Digite seu salário: R$ '))
while salario < 0:
    salario = float(input('Salário inválido! Digite um valor maior ou igual a zero: R$ '))

# Exibe os dados somente após todas as validações
print('\n' + '-' * 50)
print('DADOS CADASTRADOS COM SUCESSO!')
print(f'Nome: {nome}')
print(f'Idade: {idade}')
print(f'Salário: R$ {salario:.2f}')
print('-' * 50)
```

Exercício 9

Calculadora de duas operações. Faça um programa que receba um menu, nesse menu deverá conter a opção de somar e subtrair. Também deverá criar uma opção de sair do programa.

- Se ele escolher: **Somar**. Deverá informar dois campos para receber os valores da soma, e o resultado da operação.
- Se escolher: **Subtrair**. Deverá conter os mesmos dois campos, mas com a operação de subtração.
- E se ele escolher **sair** (s/n) do programa, deverá interromper o programa.
- Lembrem-se que a cada laço o sair do programa e o menu deverá aparecer. O menu aparecerá no início do programa e o sair no fim de cada laço.

Exercício 9

Console

```
Escolha:  
    [ 1 ] Para Somar  
    [ 2 ] Para Subtrair  
Sua opção: 1  
Primeiro número: 2  
Segundo número: 5  
Soma: 2 + 5 = 7  
Sair do programa? [s]im ou [n]ão:s  
PROGRAMA FECHADO
```

Console

```
Escolha:  
    [ 1 ] Para Somar  
    [ 2 ] Para Subtrair  
Sua opção: 2  
Primeiro número: 53  
Segundo número: 5  
Subtração: 53 - 5 = 48  
Sair do programa? [s]im ou n[ao]:s  
** PROGRAMA FECHADO **
```

Exercício 9

Console

```
Escolha:
  [ 1 ] Para Somar
  [ 2 ] Para Subtrair
Sua opção: 1
Primeiro número: 2
Segundo número: 5
Soma: 2 + 5 = 7
Sair do programa? [s]im ou [n]ão:s
PROGRAMA FECHADO
```

```
Escolha:
  [ 1 ] Para Somar
  [ 2 ] Para Subtrair
Sua opção: 2
Primeiro número: 53
Segundo número: 5
Subtração: 53 - 5 = 48
Sair do programa? [s]im ou n[ao]:s
** PROGRAMA FECHADO **
```

```
sair = ''

while True:
    calcular = input('Escolha:
[ 1 ] Para Somar
[ 2 ] Para Subtrair

Sua opção: ')

    if calcular == '1':
        n1 = int(input('Primeiro número: '))
        n2 = int(input('Segundo número: '))
        soma = n1 + n2

        print(f'Soma: {n1} + {n2} = {soma}')

        # Transforma em minúsculo e verifica se começa com "s"
        sair = input('Sair do programa? [s]im ou [n]ão: ').lower().startswith('s')

    elif calcular == '2':
        n1 = int(input('Primeiro número: '))
        n2 = int(input('Segundo número: '))
        sub = n1 - n2

        print(f'Subtração: {n1} - {n2} = {sub}')

        # Transforma em minúsculo e verifica se começa com "s"
        sair = input('Sair do programa? [s]im ou [n]ão: ').lower().startswith('s')

    else:
        print('Digitou número inválido! Tente novamente!')

# "is" é um operador lógico de comparação
if sair is True:
    print('** PROGRAMA FECHADO **')

    break

else:
    continue
```